DESENVOLVIMENTO FRONT-END PARA WEB

GitHub: fundamentos de versionamento e colaboração

Roteiro prático passo a passo para a aula

Prof. Me. Gabriel Rosa Paz

Objetivo

Criar um repositório no GitHub, enviar um pequeno projeto HTML + CSS, realizar alterações, registrar commits, consultar o histórico, colaborar com outra pessoa e conhecer, de forma introdutória, branches e pull requests, e publicar no GitHub Pages.

1. Conceitos fundamentais antes da prática

Git e GitHub são relacionados, mas não são a mesma coisa. Git é o sistema de controle de versão usado para registrar mudanças em arquivos. GitHub é uma plataforma online que hospeda repositórios Git e acrescenta recursos de colaboração, revisão e gerenciamento.

Termo	Explicação simples
Repositório	Pasta do projeto acompanhada pelo sistema de versionamento.
Commit	Registro de uma alteração. Funciona como um ponto do histórico do projeto.
Branch	Linha paralela de desenvolvimento usada para trabalhar sem alterar imediatamente a versão principal.
main	Nome comum da branch principal do projeto.
Pull request	Pedido para revisar e incorporar mudanças de uma branch em outra.
Merge	Ação de juntar mudanças de uma branch em outra.
Clone	Cópia completa do repositório do GitHub para o computador.
Push	Envio de commits locais para o repositório remoto.
Pull	Atualização do repositório local com mudanças do remoto.

Modelo mental
Arquivo alterado → commit registra a mudança → histórico permite consultar versões. Branch permite trabalhar em paralelo; pull request permite propor e revisar uma mudança antes do merge.

2. Projeto que será usado na atividade

O projeto possui apenas dois arquivos:

```
projeto-conectaweb/
├── index.html
└── style.css
```

A página é propositalmente simples. O foco desta aula não é desenvolver um site complexo, e sim usar arquivos conhecidos para entender versionamento.

2.1 index.html

```
<!DOCTYPE html>
<html lang="pt-BR">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Projeto ConectaWeb</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>
```

```
    <header>
        <h1>Projeto ConectaWeb</h1>
        <p>Um site simples para praticar HTML, CSS e versionamento.</p>
    </header>

    <nav>
        <a href="#sobre">Sobre</a>
        <a href="#recursos">Recursos</a>
        <a href="#contato">Contato</a>
    </nav>

    <main>

        <section id="sobre">
            <h2>Sobre o projeto</h2>
            <p>
                Este site foi criado como atividade prática para aprender os
                fundamentos de organização de projetos e versionamento no GitHub.
            </p>
        </section>

        <section id="recursos">
            <h2>O que vamos praticar?</h2>

            <div class="cards">

                <article class="card">
                    <h3>HTML</h3>
                    <p>Estrutura semântica e organização do conteúdo.</p>
                </article>

                <article class="card">
                    <h3>CSS</h3>
                    <p>Cores, espaçamentos e apresentação visual.</p>
                </article>

                <article class="card">
                    <h3>GitHub</h3>
                    <p>Repositórios, commits, branches e colaboração.</p>
                </article>

            </div>
        </section>

        <section id="contato">
            <h2>Contato</h2>
            <p>E-mail: contato@exemplo.com</p>
        </section>

    </main>

    <footer>
        <p>Desenvolvimento Front-End para Web - Atividade GitHub</p>
    </footer>

</body>

</html>
```

2.2 style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f4f7;
    color: #222222;
    margin: 0;
}

header {
    background-color: #1f4e78;
    color: white;
    text-align: center;
    padding: 30px 20px;
}

nav {
    background-color: #173a5b;
    text-align: center;
    padding: 12px;
}

nav a {
    color: white;
    text-decoration: none;
    margin: 0 12px;
}

nav a:hover {
    text-decoration: underline;
}

main {
    max-width: 1000px;
    margin: 0 auto;
    padding: 25px;
}

section {
    margin-bottom: 35px;
}

.cards {
    display: flex;
    gap: 20px;
    flex-wrap: wrap;
}

.card {
    flex: 1 1 220px;
    background-color: white;
    border: 1px solid #dddddd;
    border-radius: 10px;
    padding: 20px;
    box-shadow: 0 3px 10px rgba(0, 0, 0, 0.10);
}

.card h3 {
    color: #1f4e78;
}

footer {
    background-color: #1f4e78;
    color: white;
    text-align: center;
```

```
    padding: 18px;
}

@media (max-width: 600px) {

    nav a {
        display: block;
        margin: 8px 0;
    }

    .cards {
        flex-direction: column;
    }

}
```

3. Etapa 1 - Criar o repositório

Cada aluno ou grupo deverá entrar em sua conta no GitHub.

1. No GitHub, use o botão de criação no canto superior direito e escolha New repository.
2. No campo Repository name, use um nome simples, por exemplo: projeto-conectaweb.
3. Em Description, escreva uma descrição curta, por exemplo: Atividade prática de GitHub da disciplina Desenvolvimento Front-End para Web.
4. Escolha a visibilidade: Public ou Private.
5. Para esta atividade, marque Add a README file. O README será útil para descrever o projeto.
6. Clique em Create repository.

Público x privado
Public: qualquer pessoa pode visualizar o conteúdo do repositório. Private: apenas o proprietário e pessoas autorizadas podem acessar. Em um repositório pessoal, colaboradores convidados podem escrever no repositório depois de aceitar o convite.

4. Etapa 2 - Convidar um colaborador

Para demonstrar trabalho em grupo, o proprietário poderá convidar um colega.

7. Abra o repositório.
8. Clique em Settings. Se a guia não estiver visível, procure-a no menu adicional do repositório.
9. Na seção Access, escolha Collaborators.
10. Clique em Add people.
11. Pesquise pelo nome de usuário ou e-mail da pessoa.
12. Selecione a pessoa e confirme o convite.
13. O convidado deverá aceitar o convite antes de colaborar.

Atenção
Não compartilhe senha da conta. Cada integrante utiliza sua própria conta do GitHub. A colaboração é concedida pelo convite ao repositório.

5. Etapa 3 - Enviar os arquivos manualmente

Hoje faremos o primeiro envio diretamente pelo navegador.

14. Na página principal do repositório, abra o menu Add file.

15. Escolha Upload files.
16. Arraste index.html e style.css para a área indicada ou use a opção para escolher os arquivos.
17. Revise a lista de arquivos antes de confirmar.
18. No campo de mensagem do commit, escreva algo claro, por exemplo: "Adiciona estrutura inicial do site".
19. Confirme o commit.

O que acabou de acontecer?
O GitHub adicionou os arquivos ao repositório e criou um commit. O commit registra quem realizou a alteração, quando ela ocorreu e quais arquivos foram modificados.

5.1 Outras formas de enviar arquivos

Hoje usaremos o navegador, mas em projetos reais é comum trabalhar localmente.

Forma	Ideia
Navegador	Upload manual e edição direta no GitHub.
Linha de comando	Uso de git add, git commit e git push.
VS Code	Integração gráfica com Git e GitHub dentro do editor.
GitHub Desktop	Aplicativo gráfico para trabalhar com repositórios Git.

Exemplo apenas para reconhecimento, sem necessidade de executar hoje:

```
git add .
git commit -m "Atualiza página inicial"
git push origin main
```

6. Etapa 4 - Editar um arquivo diretamente no GitHub

Vamos criar uma nova versão do site sem baixar o projeto.

20. Abra index.html dentro do repositório.
21. Clique no ícone de edição (lápis).
22. Localize o título <h1>Projeto ConectaWeb</h1>.
23. Troque para: <h1>Projeto ConectaWeb - Turma ADS</h1>.
24. Clique em Commit changes.
25. Escreva uma mensagem como: "Atualiza título da página".
26. Nesta primeira demonstração, faça o commit na branch atual.

Boa mensagem de commit
Prefira mensagens que expliquem a mudança: "Atualiza cor do cabeçalho", "Adiciona seção de contato", "Corrige texto do rodapé". Evite mensagens vagas como "teste", "mudança" ou "coisas".

7. Pequenas alterações para os alunos praticarem commits

Faça uma alteração por vez e crie um commit separado para cada exercício.

Exercício	Arquivo	Mensagem sugerida
Trocar o texto do rodapé	index.html	Atualiza texto do rodapé
Adicionar mais um parágrafo em	index.html	Adiciona descrição do projeto

Sobre		
Trocar a cor do header	style.css	Atualiza cor do cabeçalho
Alterar border-radius dos cards	style.css	Ajusta arredondamento dos cards
Adicionar efeito hover aos cards	style.css	Adiciona efeito hover nos cards

Exemplo de hover que pode ser incluído:

```
.card {
    transition: transform 0.3s;
}

.card:hover {
    transform: translateY(-5px);
}
```

Por que fazer vários commits?
Porque o histórico fica mais claro. Cada commit representa uma mudança pequena e compreensível, facilitando saber o que foi alterado e quando.

8. Etapa 5 - Consultar o histórico de versões

Um dos principais objetivos do controle de versão é manter o histórico. Depois de criar alguns commits, compare as mudanças.

27. Volte à página principal do repositório.
28. Abra a lista de commits/histórico disponível na área do repositório.
29. Observe a mensagem, o autor e a data de cada commit.
30. Abra um commit para visualizar o diff.
31. Observe as linhas removidas e adicionadas.
32. Abra novamente index.html e consulte o histórico específico do arquivo, quando disponível na interface.

Diff
Diff significa diferença. Ele mostra o que mudou entre versões. É uma das ferramentas mais importantes para entender a evolução de um projeto.

9. Etapa 6 - Entendendo branch

Uma branch é uma linha paralela de desenvolvimento. Ela permite experimentar uma alteração sem modificar imediatamente a branch principal.

```
main
  |
  |---- commit A
  |---- commit B
  |
  +---- nova-cor
          |
          +---- commit C
```

Vamos criar uma branch apenas para experimentar uma alteração visual.

33. Na página do repositório, clique no seletor de branch que normalmente mostra main.

34. Digite um nome, por exemplo: nova-cor.
35. Escolha a opção para criar a branch a partir de main.
36. Confirme que agora você está trabalhando em nova-cor.
37. Abra style.css e altere a cor do header.
38. Crie um commit com a mensagem: "Testa nova cor do cabeçalho".

Importante
A mudança existe na branch nova-cor, mas ainda não foi incorporada à main. Essa separação permite testar e revisar antes de alterar a versão principal.

10. Etapa 7 - Criar um Pull Request

O pull request é uma proposta para incorporar as mudanças de uma branch em outra. Ele permite comparar, discutir e revisar as alterações.

39. Abra a guia Pull requests.
40. Clique em New pull request.
41. Use main como branch base e nova-cor como branch de comparação.
42. Observe o diff e confira se a mudança é a esperada.
43. Crie o pull request com um título, por exemplo: "Atualiza cor do cabeçalho".
44. Leia a página do pull request e observe arquivos alterados, commits e discussão.
45. Na demonstração, faça o merge do pull request.
46. Volte à branch main e confirme que a alteração agora faz parte dela.

Fluxo resumido
Criar branch → alterar arquivo → commit → abrir pull request → revisar → merge. Esse fluxo é uma base importante para colaboração em projetos de software.

11. README: a apresentação do repositório

README.md é normalmente o arquivo usado para explicar o projeto. Como marcamos Add a README file ao criar o repositório, ele já existe.

Edite o README e use um conteúdo simples:

```
# Projeto ConectaWeb

Atividade prática da disciplina Desenvolvimento Front-End para Web.

## Tecnologias

- HTML5
- CSS3

## Objetivo

Praticar organização de projeto, versionamento e colaboração com GitHub.
```

Markdown
README.md usa Markdown, uma sintaxe simples de formatação de texto. O símbolo # cria um título e o hífen cria itens de lista.

12. Conceitos importantes que ficam para próximas aulas

Conceito	Hoje	Depois
Clone	Entender a ideia	Clonar o repositório no computador
Push	Entender a ideia	Enviar commits locais ao GitHub
Pull	Entender a ideia	Atualizar a cópia local
Merge conflict	Apenas mencionar	Resolver conflitos entre alterações
.gitignore	Apenas mencionar	Evitar versionar arquivos desnecessários ou sensíveis
Issues	Apenas mencionar	Organizar tarefas, bugs e melhorias

Segurança
Nunca envie senhas, tokens, chaves de API ou outros segredos para um repositório, mesmo que ele seja privado. Arquivos sensíveis devem ser tratados com práticas próprias de segurança e exclusões adequadas.

13. Desafio final da atividade

Em duplas ou grupos pequenos, conclua o roteiro abaixo:

☐ Criar o repositório.

☐ Definir se será público ou privado.

☐ Adicionar README.

☐ Convidar pelo menos um colaborador.

☐ Enviar index.html e style.css.

☐ Criar pelo menos 3 commits com mensagens claras.

☐ Consultar o histórico e abrir um commit.

☐ Criar uma branch chamada melhoria-visual.

☐ Alterar uma propriedade CSS nessa branch.

☐ Criar um pull request.

☐ Revisar o diff.

☐ Fazer o merge na main.

☐ Publicar o site com GitHub Pages e abrir o endereço no navegador.

14. Checklist de compreensão

47. O que é um repositório?
48. Qual é a diferença entre Git e GitHub?
49. O que um commit registra?
50. Por que mensagens de commit claras são importantes?
51. Para que serve uma branch?
52. O que é um pull request?
53. O que significa merge?
54. Qual é a diferença entre repositório público e privado?

55. Por que não devemos armazenar senhas e tokens no repositório?

15. Referências oficiais para consulta

- GitHub Docs - About repositories
- GitHub Docs - Hello World
- GitHub Docs - Adding a file to a repository
- GitHub Docs - Editing files
- GitHub Docs - Inviting collaborators to a personal repository

Observação: a interface do GitHub pode sofrer pequenas alterações de posição ou nomenclatura. Os conceitos de repositório, commit, branch e pull request permanecem os mesmos.

16. Etapa adicional - Publicar o site com GitHub Pages

Depois de versionar e revisar o projeto, podemos transformar o repositório em um site publicado na internet usando o GitHub Pages. Para esta atividade, como o projeto é apenas HTML e CSS, usaremos a forma mais simples: publicar diretamente a partir da branch main.

O que é GitHub Pages?
GitHub Pages é um serviço do GitHub para hospedar sites estáticos diretamente a partir de um repositório. Ele funciona muito bem para páginas feitas com HTML, CSS e JavaScript executado no navegador.

16.1 Preparar o repositório

Antes de ativar o Pages, confira:

☐ O arquivo principal chama-se index.html.

☐ index.html está na raiz do repositório, junto com style.css.

☐ O HTML usa caminhos relativos, por exemplo: href="style.css".

☐ As alterações que desejamos publicar já estão na branch main.

Para contas GitHub Free
Para esta atividade introdutória, prefira um repositório público. No GitHub Free, o GitHub Pages está disponível para repositórios públicos; a publicação a partir de repositórios privados depende do plano utilizado.

16.2 Ativar o GitHub Pages

56. Abra o repositório projeto-conectaweb.
57. Clique em Settings.
58. Na barra lateral, na área Code, planning, and automation, clique em Pages.
59. Em Build and deployment, em Source, selecione Deploy from a branch.
60. Em Branch, selecione main.
61. Na pasta, selecione /(root), pois index.html está na raiz do repositório.
62. Clique em Save.
63. Aguarde a publicação. O GitHub mostrará o endereço do site na própria página Settings > Pages quando ele estiver disponível.

Endereço esperado
Para um repositório de projeto chamado projeto-conectaweb, o endereço padrão segue normalmente o formato: https://SEU-USUARIO.github.io/projeto-conectaweb/

Existe também o site de usuário, criado em um repositório especial chamado SEU-USUARIO.github.io, cujo endereço padrão é https://SEU-USUARIO.github.io/.

16.3 Testar o site publicado

64. Abra o endereço fornecido pelo GitHub Pages em uma nova guia.
65. Confira se o título, o menu, os cards e o rodapé aparecem corretamente.
66. Reduza a largura da janela e teste a media query do projeto.
67. Se o CSS não carregar, confira se style.css está no repositório e se o caminho usado em href está correto.

16.4 Relacionando commit e publicação

Ao publicar a partir da branch main, novas alterações nessa branch podem gerar uma nova implantação do GitHub Pages. Isso ajuda a visualizar a relação entre desenvolvimento, versionamento e publicação:

```
Editar arquivo
      ↓
Criar commit
      ↓
Atualizar a branch main
      ↓
GitHub Pages publica a nova versão
      ↓
Site atualizado na internet
```

Observação
A atualização não é necessariamente instantânea. A documentação do GitHub informa que a publicação pode levar alguns minutos; em alguns casos, até cerca de 10 minutos.

16.5 Pequeno exercício

Depois que o site estiver publicado:

☐ Abra index.html diretamente no GitHub e altere uma frase visível da página.

☐ Crie um commit com uma mensagem clara, por exemplo: Atualiza mensagem da página inicial.

☐ Aguarde alguns minutos e atualize o endereço do GitHub Pages.

☐ Confirme que a nova versão foi publicada.

Atenção à privacidade
Um site do GitHub Pages deve ser tratado como conteúdo publicado na internet. Não coloque senhas, tokens, chaves de API, dados pessoais sensíveis ou qualquer informação que não deveria ser exposta.

Problema	O que conferir
Página não aparece	Confira Settings > Pages, a branch main e a pasta /(root).
CSS não carrega	Confira o nome style.css e o href usado em index.html.
Erro 404	Confira a URL e se index.html está na fonte publicada.
Versão antiga	Aguarde a implantação e atualize a página novamente.

16.6 Referência oficial

GitHub Docs - Configuring a publishing source for your GitHub Pages site.

GitHub Docs - What is GitHub Pages?